

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

доцент, канд. техн. наук

должность, уч. степень, звание

подпись, дата

А. В. Бржезовский

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ № 7

ТРИГГЕРЫ

по курсу:

МЕТОДЫ И СРЕДСТВА ПРОЕКТИРОВАНИЯ ИНФОРМАЦИОННЫХ
СИСТЕМ И ТЕХНОЛОГИЙ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4326

подпись, дата

Г. С. Томчук

инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Цель работы: изучение механизмов триггеров в MySQL для обеспечения активной целостности данных и автоматизации реакций на изменения в базе.

2 Постановка задачи

Требуется реализовать триггеры на операции INSERT, UPDATE и DELETE, обеспечивающие контроль целостности данных, а также триггер для ведения истории изменений.

Вариант: 4.

Создайте базу данных для хранения следующих сведений: студент, группа, дисциплина, преподаватель, лабораторная работа, рейтинг за сданную лабораторную работу.

3 Ход выполнения

В ходе выполнения работы была использована ранее созданная база данных с сущностями студентов, групп, дисциплин, лабораторных работ и оценок.

Были разработаны триггеры, обеспечивающие контроль целостности при добавлении, изменении и удалении записей. На рисунке 1 приведен запрос на создание таблицы для логирования изменений в таблице student и код триггера AFTER INSERT для таблицы student.

```
1  USE university_db;
2
3  ●-- таблица истории изменений student
4  DROP TABLE IF EXISTS student_audit;
5  ●CREATE TABLE student_audit (
6      audit_id INT AUTO_INCREMENT PRIMARY KEY,
7      student_id INT,
8      old_last VARCHAR(50),
9      new_last VARCHAR(50),
10     old_group_id INT,
11     new_group_id INT,
12     action_type VARCHAR(20),
13     changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
14 );
15
16 -- AFTER INSERT триггер
17 DELIMITER //
18 ●CREATE TRIGGER trg_student_after_insert
19 AFTER INSERT ON student
20 FOR EACH ROW
21 BEGIN
22     INSERT INTO student_audit(student_id, new_last, new_group_id, action_type)
23     VALUES (NEW.student_id, NEW.last_name, NEW.group_id, 'INSERT');
24 END //
25 DELIMITER ;
26
```

Рисунок 1 — Таблица изменений student и триггер, запускающийся после INSERT в таблицу student

На рисунке 2 приведен код триггера AFTER UPDATE для таблицы студентов.

```
27 -- AFTER UPDATE триггер
28 DELIMITER //
29 CREATE TRIGGER trg_student_after_update
30 AFTER UPDATE ON student
31 FOR EACH ROW
32 BEGIN
33     INSERT INTO student_audit(student_id, old_last, new_last, old_group_id, new_group_id, action_type)
34     VALUES (OLD.student_id, OLD.last_name, NEW.last_name, OLD.group_id, NEW.group_id, 'UPDATE');
35 END //
36 DELIMITER ;
```

Рисунок 2 — Триггер, запускающийся после UPDATE в таблице student

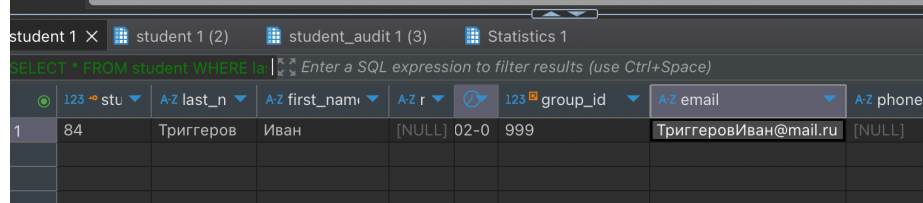
На рисунке 3 приведен код триггера AFTER DELETE для таблицы студентов и результат выполнения.

```
37 -- AFTER DELETE триггер
38 DELIMITER //
39 CREATE TRIGGER trg_student_after_delete
40 AFTER DELETE ON student
41 FOR EACH ROW
42 BEGIN
43     INSERT INTO student_audit(student_id, old_last, old_group_id, action_type)
44     VALUES (OLD.student_id, OLD.last_name, OLD.group_id, 'DELETE');
45 END //
46 DELIMITER ;
```

Рисунок 3 — Триггер, запускающийся после DELETE в таблице student

После этого были добавлены триггеры BEFORE INSERT и BEFORE UPDATE, обеспечивающие активную целостность данных. При вставке студента без электронной почты автоматически формируется значение email, а при попытке обновления записи со значением NULL электронной почты сохраняется предыдущее значение. На рисунке 4 приведен код триггера BEFORE INSERT и результат его выполнения.

```
48 -- контроль целостности: запрет пустого email (INSERT)
49 DROP TRIGGER IF EXISTS trg_student_email_check_insert;
50 DELIMITER //
51 CREATE TRIGGER trg_student_email_check_insert
52 BEFORE INSERT ON student
53 FOR EACH ROW
54 BEGIN
55     IF NEW.email IS NULL THEN
56         SET NEW.email = CONCAT(NEW.last_name, NEW.first_name, '@mail.ru');
57     END IF;
58 END //
59 DELIMITER ;
```



	student_id	last_name	first_name	group_id	email	phone
1	84	Триггеров	Иван	[NULL] 02-0 999	ТриггеровИван@mail.ru	[NULL]

Рисунок 4 — Триггер BEFORE INSERT. В результате email со значением NULL был записан как «ТриггеровИван@mail.ru»

На рисунке 5 приведен код триггера BEFORE UPDATE и результат его выполнения.

```
62 -- контроль целостности: запрет пустого email (UPDATE)
63 DELIMITER //
64 CREATE TRIGGER trg_student_email_check_update
65 BEFORE UPDATE ON student
66 FOR EACH ROW
67 BEGIN
68     IF NEW.email IS NULL THEN
69         SET NEW.email = OLD.email;
70     END IF;
71 END //
72 DELIMITER ;
73
```

student 1 | student 1 (2) X | student_audit 1 (3) | Statistics 1

SELECT * FROM student WHERE last_name = 'Обновленный' | Enter a SQL expression to filter results (use Ctrl+Space)

	128	A-Z last_name	A-Z first_name	birth_date	123 g	A-Z email	A-Z phone
	84	Обновленный	Иван	[NULL]	2002-01-01	999	ТриггеровИван@mail.ru
							[NULL]

Рисунок 5 — Триггер BEFORE UPDATE. В результате новый email со значением NULL был проигнорирован

Также был реализован триггер BEFORE DELETE, запрещающий удаление единственного студента из группы, что предотвращает нарушение логической целостности данных. На рисунке 7 приведен код триггера BEFORE DELETE и результат его выполнения.

```
73
74 -- контроль удаления: нельзя удалить единственного студента группы
75 DROP TRIGGER IF EXISTS trg_student_delete_restrict;
76 DELIMITER //
77 CREATE TRIGGER trg_student_delete_restrict
78 BEFORE DELETE ON student
79 FOR EACH ROW
80 BEGIN
81     DECLARE cnt INT;
82
83     SELECT COUNT(*) INTO cnt
84     FROM student
85     WHERE group_id = OLD.group_id;
86
87     IF cnt = 1 THEN
88         SIGNAL SQLSTATE '45000'
89         SET MESSAGE_TEXT = 'Нельзя удалить единственного студента группы';
90     END IF;
91 END //
92 DELIMITER ;
93
```

student 1 X

DELETE FROM student WHERE last_name = 'Обновленный' | Enter a SQL expression to filter results (use Ctrl+Space)

SQL Error [1644] [45000]: Нельзя удалить единственного студента группы

```
1 DELETE FROM student
2 WHERE last_name = 'Обновленный'
```

Рисунок 6 — Триггер BEFORE DELETE. В результате попытки удаления возникла ошибка

В завершение были выполнены SQL-операторы INSERT, UPDATE и DELETE, вызывающие автоматическое срабатывание разработанных триггеров. На рисунке 7 представлена таблица student_audit с залогированными действиями.

```
112  -- просмотр истории
113  SELECT * FROM student_audit;
```

Рисунок 7 — Таблица student_audit после манипуляций с таблицей student

4 Выводы

В ходе выполнения работы были изучены и реализованы триггеры в реляционной базе данных. Были разработаны триггеры для операций вставки, обновления и удаления, обеспечивающие автоматическую фиксацию изменений и контроль целостности данных.

Дополнительно реализован триггер аудита, позволяющий вести историю изменений записей. Получен практический опыт использования триггеров для реализации активной бизнес-логики на уровне СУБД и автоматического реагирования системы на изменения данных.

ПРИЛОЖЕНИЕ

```
USE university_db;

-- таблица истории изменений student
DROP TABLE IF EXISTS student_audit;
CREATE TABLE student_audit (
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id INT,
    old_last VARCHAR(50),
    new_last VARCHAR(50),
    old_group_id INT,
    new_group_id INT,
    action_type VARCHAR(20),
    changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- AFTER INSERT триггер
DELIMITER //
CREATE TRIGGER trg_student_after_insert
AFTER INSERT ON student
FOR EACH ROW
BEGIN
    INSERT INTO student_audit(student_id, new_last, new_group_id, action_type)
    VALUES (NEW.student_id, NEW.last_name, NEW.group_id, 'INSERT');
END //
DELIMITER ;

-- AFTER UPDATE триггер
DELIMITER //
CREATE TRIGGER trg_student_after_update
AFTER UPDATE ON student
FOR EACH ROW
BEGIN
    INSERT INTO student_audit(student_id, old_last, new_last, old_group_id,
    new_group_id, action_type)
    VALUES (OLD.student_id, OLD.last_name, NEW.last_name, OLD.group_id, NEW.group_id,
    'UPDATE');
END //
DELIMITER ;

-- AFTER DELETE триггер
DELIMITER //
CREATE TRIGGER trg_student_after_delete
AFTER DELETE ON student
FOR EACH ROW
BEGIN
    INSERT INTO student_audit(student_id, old_last, old_group_id, action_type)
    VALUES (OLD.student_id, OLD.last_name, OLD.group_id, 'DELETE');
END //
DELIMITER ;

-- контроль целостности: запрет пустого email (INSERT)
DROP TRIGGER IF EXISTS trg_student_email_check_insert;
DELIMITER //
CREATE TRIGGER trg_student_email_check_insert
BEFORE INSERT ON student
FOR EACH ROW
BEGIN
    IF NEW.email IS NULL THEN
        SET NEW.email = CONCAT(NEW.last_name, NEW.first_name, '@mail.ru');
    END IF;
END IF;
```

```

END //
DELIMITER ;

-- контроль целостности: запрет пустого email (UPDATE)
DELIMITER //
CREATE TRIGGER trg_student_email_check_update
BEFORE UPDATE ON student
FOR EACH ROW
BEGIN
    IF NEW.email IS NULL THEN
        SET NEW.email = OLD.email;
    END IF;
END //
DELIMITER ;

-- контроль удаления: нельзя удалить единственного студента группы
DROP TRIGGER IF EXISTS trg_student_delete_restrict;
DELIMITER //
CREATE TRIGGER trg_student_delete_restrict
BEFORE DELETE ON student
FOR EACH ROW
BEGIN
    DECLARE cnt INT;

    SELECT COUNT(*) INTO cnt
    FROM student
    WHERE group_id = OLD.group_id;

    IF cnt = 1 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Нельзя удалить единственного студента группы';
    END IF;
END //
DELIMITER ;

-- тестовые данные для вызова триггеров
INSERT INTO student_group(group_id, group_name, admission_year)
VALUES (999, 4999, 2024);

-- DELETE FROM student WHERE group_id = 999;
INSERT INTO student(last_name, first_name, birth_date, group_id, email)
VALUES ('Триггеров', 'Иван', '2002-01-01', 999, NULL);

SELECT * FROM student WHERE last_name = 'Триггеров';

UPDATE student
SET last_name = 'Обновленный', email = NULL
WHERE last_name = 'Триггеров';
SELECT * FROM student WHERE last_name = 'Обновленный';

DELETE FROM student
WHERE last_name = 'Обновленный';

-- просмотр истории
SELECT * FROM student_audit;

```